

● Classical/Modelling ● PID ● Frequency ● State Space ● Observer ● LQR ● Exam Logic

Observer Design & Separation ●
Let $e = x - \hat{z}$: $\dot{e} = (A - LC)e$.
Choose L so $A - LC$ is stable and suitably fast; observability is required for arbitrary observer pole placement. By duality,
$L = \text{place}(A^T, C^T, p_{\text{obs}})^T$.
Rule of thumb: observer poles about 2–5 times faster than controller poles. Too slow $\rightarrow$ delayed/poor estimates. Too fast $\rightarrow$ large L , noise amplification, actuator activity and model-order sensitivity.
Separation principle:
$\text{eig}(A_{\text{combined}}) = \text{eig}(A - BK) \cup \text{eig}(A - LC)$.

Thus K and L may be designed independently, then combined with $u = -K\hat{x} + N\dot{r}$.

Q6: LQR Core Workflow ●

$$J = \int_0^\infty \left(x^T Q x + u^T R u \right) dt, \quad u^* = -Kx$$

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

$$K = R^{-1} B^T P.$$

Requirements for standard infinite-horizon result: $Q \succeq 0$, $R \succ 0$, (A, B) stabilisable and penalised unstable modes detectable. **Matrix procedure:** choose Q, R ; assume symmetric P for hand calculation; substitute into ARE; equate matrix entries; select stabilising $P \succeq 0$; calculate K ; verify $\text{eig}(A - BK)$.

LQR Weight Logic / Bryson's Rule ●

$Q_{ii} \uparrow$ state x_i is expensive: tighter/faster regulation, usually higher control effort. $R_{jj} \uparrow$ input u_j is expensive: smaller/conservative actuation, usually slower response. Only the relative scale of Q and R matters strongly. For systematic initial weights:

$$Q_{ii} = \frac{1}{(x_{i,\text{max}})^2}, \quad R_{jj} = \frac{1}{(u_{j,\text{max}})^2}.$$

Then tune iteratively. Increasing R is the expected short-answer response to “what reduces control effort?”

LQR: Advantages, Robustness, Limits ● **Why optimal?** K minimises the stated quadratic cost among admissible controls. Closed-loop poles emerge from the performance/effort trade-off rather than being assigned directly.

Advantages over PID: natural MIMO/state coupling; systematic matrix design; explicit control effort trade-off; full-state SISO LQR guarantees at plant input; gain margin at least $[0.5, \infty)$ (-6 dB to $+\infty$) and phase margin at least 60° . **PID advantages:** simple, inexpensive, familiar, often needs less model/state information, easy for SISO loops. **LQR limitations:** linear model, full state required, Q/R tuning still heuristic, no integral action by default, constraints/saturation not handled explicitly. Adding an observer gives LQG later, but the full-state LQR margin guarantees may be lost.

Pole Placement vs LQR ●

	Pole placement	LQR
Design input	desired poles	Q, R weights
Result	exact poles	poles emerge opt- mally
Cost	none explicit	minimises J
MIMO	possible but choices	natural matrix
Robustness	no inherent guaran-	full-state input-
Main risk	aggressive arbitrary poles	margin guarantees model/state de-
		pendence; weight tuning

Why not always place poles very far left? The response may be fast mathematically but require unrealistic gains, saturate actuators and amplify noise/unmodelled dynamics.

2 x 2 Matrix Mini-Example ●

Given

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Controllability: $AB = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$, $C = \begin{bmatrix} 0 & 1 \\ 1 & -3 \end{bmatrix}$,
 $\det C = -1 \neq 0$.

Observability: $CA = \begin{bmatrix} 0 & 1 \end{bmatrix}$, $O = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, rank = 2.

Pole placement: let $K = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$. Then

$$\det(sI - A + BK) = s^2 + (3 + k_2)s + (2 + k_1).$$

Desired poles $-4, -5$ give $s^2 + 9s + 20$, hence $K = \begin{bmatrix} 18 & 6 \end{bmatrix}$.

Observer: let $L = \begin{bmatrix} l_1 \\ l_2 \end{bmatrix}$. Then

$$\det(sI - A + LC) = s^2 + (l_1 + 3)s + (3l_1 + l_2 + 2).$$

Desired poles $-8, -9$ give $s^2 + 17s + 72$, hence $L = \begin{bmatrix} 14 \\ 28 \end{bmatrix}$.

2 x 2 ARE Hand Pattern ●

For the double integrator

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}, \quad R = r,$$

assume symmetric $P = \begin{bmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{bmatrix}$. The ARE gives

$$q_1 - \frac{p_{12}^2}{r} = 0, \quad p_{11} - \frac{p_{12}p_{22}}{r} = 0,$$

$$q_2 + 2p_{12} - \frac{p_{22}^2}{r} = 0.$$

Choose the stabilising positive-semidefinite solution and then

$$K = R^{-1} B^T P = \frac{1}{r} \begin{bmatrix} p_{12} & p_{22} \end{bmatrix}.$$

This is the common hand-calculation pattern: substitute symmetric P , equate entries, select the physically valid/stabilising root.

Matrix Calculation Checklist ●

Rank: row-reduce; rank is number of pivots. For square matrices, nonzero determinant implies full rank. **Eigenvalues:** solve $\det(AI - A) = 0$. Continuous stability requires $\text{Re } \lambda < 0$; discrete stability requires $|\lambda| < 1$. **2 x 2 inverse:**

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Matrix order: AB exists when inner dimensions match; generally $AB \neq BA$. **Rank:** row-reduce; rank is number of pivots. For square matrices, nonzero determinant implies full rank. **Eigenvalues:** solve $\det(AI - A) = 0$. Continuous stability requires $\text{Re } \lambda < 0$; discrete stability requires $|\lambda| < 1$. **2 x 2 inverse:**

High-Probability One-Sentence Answers ● **Why feedback?** It compares measured output with the reference so disturbances and modelling errors can be corrected. **Why integral action?** It accumulates persistent error and raises system type, removing step steady-state error, but can reduce stability margins. **Why derivative filtering?** Derivative action amplifies high-frequency measurement noise, so a low-pass filter is required.

Why controllability first? State feedback cannot move an uncontrollable mode. **Why observability first?** An observer cannot re-construct an unobservable mode from the available outputs. **Observer role?** It estimates unavailable states so state-feedback control can be implemented from measured inputs and outputs. **Large K in LQR?** Control effort is penalised more, so the controller becomes less aggressive and usually slower. **Lead or lag?** Use lead for transient response/phase margin; lag for steady-state accuracy/low-frequency gain.

Integrated Design Workflow ● 1. Derive a model and state assumptions/valid range. 2. Identify poles, stability and step-response limitations. 3. For simple SISO: PID/lead-lag; check Bode/Nyquist margins and actuator nonlinearities. 4. For state-space/MIMO: check controllability and observability. 5. Choose pole placement for exact dynamics or LQR for a performance-effort optimum. 6. Add an observer when states are not measured; avoid unnecessarily fast observer poles. 7. Add prefilter/integral augmentation for tracking. 8. Validate noise, delay, saturation, uncertainty and sampling—a controller that works only for the nominal linear model is not hardware-ready.

Q7: Kalman Filter — Purpose & Noise Logic ●

Stochastic model:

$$x_{k+1} = Ax_k + Bu_k + Gw_k, \quad y_k = Cx_k + Du_k + v_k.$$

Process noise w : disturbances/unmodelled dynamics. Measurement noise v : sensor noise/quantisation. Standard assumptions: zero-mean, white, Gaussian and mutually uncorrelated.

$$Q_w = E[ww^T] \succeq 0, \quad R_v = E[vv^T] \succ 0.$$

Purpose: recursively estimate unmeasured states while minimising the estimation-error covariance

$$P = E[(x - \hat{x})(x - \hat{x})^T].$$

Kalman vs Luenberger: a Luenberger observer places poles deterministically; a Kalman filter chooses the gain optimally from model and noise covariances. **Trust rule:** larger $R_v \Rightarrow$ trust sensors less/model more; larger $Q_w \Rightarrow$ model is less trusted and measurements receive more weight.

Kalman Matrix Workflow — Predict then Update ●

- Prediction (a priori):**

$$\hat{x}_k^- = A\hat{x}_{k-1} + B u_{k-1},$$

$$P_k^- = A P_{k-1} A^T + G Q_w G^T.$$
- Innovation and gain:**

$$\hat{y}_k = y_k - C \hat{x}_k^- - D u_k,$$

$$S_k = C P_k^- C^T + R_v, \quad L_k = P_k^- C^T S_k^{-1}.$$
- Correction (a posteriori):**

$$\hat{x}_k = \hat{x}_k^- + L_k \hat{y}_k, \quad P_k = (I - L_k C) P_k^-.$$

Meaning: prediction propagates both state and uncertainty; the update corrects the model using the measurement residual. P_k usually grows between measurements because process noise adds uncertainty.

Steady-State Kalman Filter & Duality ● **Continuous observer:**

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}).$$

At each sample:

- measure/estimate current state x_k ;
- predict behaviour over N future steps;
- minimise tracking error and control effort subject to constraints;
- apply only the first optimal input u_k^* ;
- shift the horizon and solve again using new measurements.

Typical finite-horizon cost:

$$J = \sum_{i=0}^{N-1} \left(e_i^T Q e_i + u_i^T R u_i \right) + e_N^T P_f e_N,$$

$$e_i = x_i - r_i.$$

For smoother actuators, penalise Δu_i instead of/in addition to u_i . Future references $r_{k+1:k+N}$ give preview, so MPC can anticipate turns or setpoint changes.

MPC Quadratic Program & Constraints ●

Stack future inputs:

$$U = \begin{bmatrix} u_k^T & u_{k+1}^T & \cdots & u_{k+N-1}^T \end{bmatrix}^T.$$

After substituting predictions into the quadratic cost:

$$\min_U \frac{1}{2} U^T H U + f^T U$$

subject to

$$A_{\text{ineq}} U \leq b_{\text{ineq}}.$$

Constraints may represent

$$u_{\min} \leq u_i \leq u_{\max}, \quad \Delta u_{\min} \leq \Delta u_i \leq \Delta u_{\max},$$

$$x_{\min} \leq x_i \leq x_{\max}, \quad y_{\min} \leq y_i \leq y_{\max}.$$

$H \succeq 0$ gives a convex QP. Hard constraints must never be violated; soft constraints add slack variables and penalties to preserve feasibility.

Failure modes: infeasible QP, excessive computation, model mismatch and delay. Use suitable horizon/sample time, terminal ingredients, warm starts and fallback control.

MPC vs LQR — Likely Comparison Question ●●

	LQR	MPC
Horizon	infinite	finite, repeatedly shifted
Constraints	not explicit	state/input/rate constraints
Control	$u = -Kx$	online optimisation
Computation	offline; cheap on-line	QP every sample
Model	normally LTI	LTI/LTV; NMPC possible
Stability	Riccati guarantee	needs suitable terminal design
Preview	limited	uses future references

Connection: unconstrained MPC with a sufficiently long/infinite horizon and correct terminal cost approaches the LQR solution. **Why choose MPC?** It explicitly handles physical constraints and predicts future behaviour/reference changes.

Main disadvantage? Online optimisation increases computation and can become infeasible too slow.

Adaptive Control: Gain Scheduling, MRAC, STR ●

Gain scheduling: measure an operating variable and switch/interpolate among predesigned controllers. Fast and practical for predictable variation, but essentially open-loop adaptation and transitions need validation. **MRAC:** choose a reference model with desired response and adapt controller parameters so plant output tracks it:

$$e = y - y_m, \quad u = \theta^T \phi,$$

$$\dot{\theta} = -\gamma \frac{\partial e}{\partial \theta} e.$$

MIT rule: Lyapunov laws give stronger guarantees. Direct adaptation: no explicit plant identification. **Self-tuning regulator (STR):** repeatedly identify plant parameters online, then redesign the controller using certainty equivalence. Flexible and diagnostic, but computationally heavier and requires persistent excitation. **Selection:** predictable measured variation $\rightarrow$ gain scheduling; known desired model but uncertain plant $\rightarrow$ MRAC; explicit changing model required $\rightarrow$ STR.

LQG, Separation Principle & LTR ●●

LQG: LQR state feedback plus Kalman state estimation:

$$u = -K\hat{x}, \quad \dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}).$$

Separation principle: design K and L independently; nominal closed-loop poles are

$$\text{eig}(A - BK) \cup \text{eig}(A - LC).$$

This guarantees nominal stability under the usual controllability/detectability assumptions, but it does **not** preserve LQR's guaranteed gain/phase margins. The estimator adds dynamics and phase lag, so LQG can be fragile to model error.

Loop Transfer Recovery (input LTR): (1) design LQR gain K_i ; (2) choose fictitious process-noise channel $G = B$ and $Q_w = qI$; (3) increase q . As $q \rightarrow \infty$, the LQG input-loop approaches the LQR loop and robustness margins are recovered.

Trade-off: aggressive recovery increases estimator bandwidth/gain and can amplify sensor noise or actuator activity.

Robust-Control Backup: H_∞ / Small Gain ●

$$S = (I + L)^{-1}, \quad T = L(I + L)^{-1}, \quad S + T = I.$$

S : tracking/disturbance sensitivity at low frequency. T : noise/unmodelled-dynamics sensitivity at high frequency.

$$\|M\|_\infty = \sup_\omega \sigma(M(j\omega)).$$

Small-gain theorem: feedback of stable M and uncertainty Δ is robustly stable if

$$\|M\|_\infty \|\Delta\|_\infty < 1.$$

Common weighted tests:

$$\frac{\|W_u T\|_\infty}{\|W_s\|_\infty} < 1 \quad (\text{robust stability}),$$

$$\frac{\|W_u\|_\infty}{\|W_s\|_\infty} < 1 \quad (\text{performance}).$$

Intuition: robust design limits the worst-case amplification over all frequencies, rather than optimising only the nominal model.

Q8: MPC Receding-Horizon Workflow ●

Prediction model:

$$x_{i+1} = Ax_i + Bu_i, \quad y_i = Cx_i.$$

At each sample:

- measure/estimate current state x_k ;
- predict behaviour over N future steps;
- minimise tracking error and control effort subject to constraints;
- apply only the first optimal input u_k^* ;
- shift the horizon and solve again using new measurements.

Typical finite-horizon cost:

$$J = \sum_{i=0}^{N-1} \left(e_i^T Q e_i + u_i^T R u_i \right) + e_N^T P_f e_N,$$

$$e_i = x_i - r_i.$$

For smoother actuators, penalise Δu_i instead of/in addition to u_i . Future references $r_{k+1:k+N}$ give preview, so MPC can anticipate turns or setpoint changes.

MPC Quadratic Program & Constraints ●

Stack future inputs:

$$U = \begin{bmatrix} u_k^T & u_{k+1}^T & \cdots & u_{k+N-1}^T \end{bmatrix}^T.$$

After substituting predictions into the quadratic cost:

$$\min_U \frac{1}{2} U^T H U + f^T U$$

subject to

$$A_{\text{ineq}} U \leq b_{\text{ineq}}.$$

Constraints may represent

$$u_{\min} \leq u_i \leq u_{\max}, \quad \Delta u_{\min} \leq \Delta u_i \leq \Delta u_{\max},$$

$$x_{\min} \leq x_i \leq x_{\max}, \quad y_{\min} \leq y_i \leq y_{\max}.$$

$H \succeq 0$ gives a convex QP. Hard constraints must never be violated; soft constraints add slack variables and penalties to preserve feasibility. **Failure modes:** infeasible QP, excessive computation, model mismatch and delay. Use suitable horizon/sample time, terminal ingredients, warm starts and fallback control.

MPC vs LQR — Likely Comparison Question ●●

	LQR	MPC
Horizon	infinite	finite, repeatedly shifted
Constraints	not explicit	state/input/rate constraints
Control	$u = -Kx$	online optimisation
Computation	offline; cheap on-line	QP every sample
Model	normally LTI	LTI/LTV; NMPC possible
Stability	Riccati guarantee	needs suitable terminal design
Preview	limited	uses future references

Connection: unconstrained MPC with a sufficiently long/infinite horizon and correct terminal cost approaches the LQR solution. **Why choose MPC?** It explicitly handles physical constraints and predicts future behaviour/reference changes.

Main disadvantage? Online optimisation increases computation and can become infeasible too slow.

Adaptive Control: Gain Scheduling, MRAC, STR ●

Gain scheduling: measure an operating variable and switch/interpolate among predesigned controllers. Fast and practical for predictable variation, but essentially open-loop adaptation and transitions need validation. **MRAC:** choose a reference model with desired response and adapt controller parameters so plant output tracks it:

$$e = y - y_m, \quad u = \theta^T \phi,$$

$$\dot{\theta} = -\gamma \frac{\partial e}{\partial \theta} e.$$

MIT rule: Lyapunov laws give stronger guarantees. Direct adaptation: no explicit plant identification. **Self-tuning regulator (STR):** repeatedly identify plant parameters online, then redesign the controller using certainty equivalence. Flexible and diagnostic, but computationally heavier and requires persistent excitation. **Selection:** predictable measured variation $\rightarrow$ gain scheduling; known desired model but uncertain plant $\rightarrow$ MRAC; explicit changing model required $\rightarrow$ STR.

Reinforcement Learning & ILC — Benefits/s/Risks ●

RL learns a policy from interaction to maximise discounted reward:

$$\max_{\pi} E \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Benefits: can learn nonlinear policies without an accurate model, adapt to unknown/changing dynamics and optimise long-term behaviour. **Risks:** unsafe exploration, weak stability/constraint guarantees, large data demand, reward misspecification, poor interpretability and simulation-to-real mismatch. Use offline training, safety filters, constraints and a conventional fallback controller. **ILC:** for the same finite task repeated many times,

$$u_{j+1}(t) = u_j(t) + L_e e_j(t).$$

It improves trial-to-trial feedforward performance, but is not general online adaptation to arbitrary new tasks.

Planning Essentials: A*, RRT, Hierarchy ●

Hierarchy: mission goal $\rightarrow$ collision-free path $\rightarrow$ timed trajectory $\rightarrow$ feedback control. Plan slowly; control faster; safety filtering fastest.

A*:

$$f(n) = g(n) + h(n),$$

where g is actual cost-to-come and h estimates cost-to-go. If $h(n) \leq h^*(n)$ (admissible), A* is optimal; Dijkstra is A* with $h = 0$. A consistent heuristic also satisfies

$$h(n) \leq c(n, n') + h(n').$$

RRT: randomly sample, find nearest tree node, steer along step and add the edge to collision-free. Probabilistically complete but not optimal. **RRT*** adds neighbour selection/rewiring and is asymptotically optimal. **Key distinction:** a path is geometric; a trajectory adds timing/velocity/acceleration and must be dynamically feasible.

CBF Safety & Multi-Agent Consensus ●●

Define safe set

$$C = \{x : h(x) \geq 0\}.$$

For $\dot{x} = f(x) + g(x)u$, enforce

$$L_f h(x) + L_g h(x)u + \alpha h(x) \geq 0.$$

This is linear in u and can be added to a QP safety filter that minimises deviation from a nominal PID/LQR/MPC/RL command. If initially safe and feasible, the condition makes C forward invariant. Potential fields are simpler but may have local minima and no formal safety guarantee.

Consensus protocols:

$$\dot{x}_i = - \sum_j a_{ij} (x_i - x_j), \quad \dot{x} = -\mathcal{L}x,$$

$$\mathcal{L} = D - A.$$

For a connected undirected communication graph, agents converge to a common value. Formation control applies consensus to shared states/desired relative offsets. Delays, switching topology and packet loss can degrade convergence/stability.

Q9: System-Synthesis Answer Framework ●

Use this order for a design/justify question:

- Requirements:** tracking, disturbance rejection, speed, safety, constraints, compute budget.
- Model:** TF for simple SISO insight; state-space for MIMO/internal dynamics; state assumption- s /uncertainty.
- Feasibility:** check stability, controllability and observability; identify required sensors/actuators.
- Classical baseline:** PID/lead-lag for transparent, low-cost SISO control; verify step response, GM and PM.
- Modern layer:** LQR for MIMO regulation; Kalman/LQG for noisy unmeasured states; MPC for constraints/preview; adaptive/RL only when dynamics vary or are poorly known.
- Safety/implementation:** saturation, anti-windup, filtering, sampling delay, rate limits, CBF/emergency fallback.
- Validation:** nominal + uncertainty/noise/delay tests; worst cases, Monte Carlo, HIL and fail-safe behaviour.

Strong justification form: Use method X because requirement Y matches its main strength; mitigate limitation Z using safeguard W.

Method Selection — One-Line Justifications ●

PID simple SISO plant, low computation, easy commissioning; add anti-windup/filtering.

Lead/lag reshape transient response or steady-state accuracy with classical margins.

Pole placement direct transient design when states are available and system controllable.

LQR systematic MIMO state regulation measurements.

Kalman/LQG states unmeasured and sensors noisy; then re-check robustness/LTR.

MPC hard constraints, rate limits and future reference preview are central. parameters vary predictably or online identification is required.

RL model is poor and nonlinear decision policy is valuable; requires safety supervision.

CBF hard safety set must remain invariant while preserving a nominal controller.

Never ignore: actuator saturation/dead-zone, measurement noise, sampling/computation delay, nonlinear operating range, model uncertainty, sensor failure and communication loss.

Week 3 High-Probability Answers ●

Purpose of a Kalman filter? To combine a model and noisy measurements into a minimum-variance estimate of unmeasured states.

Why can LQG be less robust than LQR? The estimator adds dynamics/phase lag, so the full-state LQR margin guarantees no longer apply.

What does LTR do? It tunes the estimator so the LQG loop approaches the robust LQR loop.

What is receding horizon? Optimise several future inputs, apply only the first, then re-solve using new measurements.

Main MPC strength? Explicit handling of state, input and rate constraints with prediction/reference preview.

RL advantage and risk? It can learn without an accurate model, but exploration may violate safety and stability is not guaranteed.

Why use a CBF? It converts a safety requirement into a control constraint that keeps the safe set forward invariant.

Planning vs control? Planning selects a feasible route/reference; feedback control tracks it and rejects disturbances.